

UNIVERSITETI I EVROPËS JUGLINDORE
УНИВЕРЗИТЕТ НА ЈУГОИСТОЧНА ЕВРОПА
SOUTH EAST EUROPEAN UNIVERSITY

FACULTY OF CONTEMPORARY SCIENCES AND TECHNOLOGIES
UNDERGRADUATE STUDIES IN THE FIELD OF COMPUTER SCIENCE

MapOut

Week 6: Constraint Satisfaction Problems and Planning

Student: Mirjana Arsenoska
Student ID: 131834

Professor: Prof. Dr. Azir Aliu
Course: Introduction to Artificial Intelligence

Tetovo, 2026

CONTENT

Introduction 1
1. Constraint Satisfaction Problems (CSP) 2
1.1 Applying Variables, Domains, and Constraints 2
1.2 Backtracking Search 3
2. CSP Implementation in MapOut 4
2.1 CSP Planner Implementation ..... **Error! Bookmark not defined.**
2.2 Generating Multi-Day Itineraries 4
3. Real Travel Time and Distance Calculation 5
3.1 Route Caching 5
4. Implementation and Testing 6
4.1 Dataset Validation 6
4.2 Testing the CSP Planner 6
4.3 Testing Real Routing 7
5. Current Limitations and Future Improvements 8
Conclusion 9
AI Assistance Disclosure 9

Introduction

During Week 6, I focused on improving the itinerary generation process in MapOut by applying Constraint Satisfaction Problems (CSP) and backtracking search. The goal was to move beyond simply selecting attractions that match the user's preferences and start considering how those attractions can be organized into a realistic travel plan.

In the previous implementation, MapOut could filter attractions based on the selected destination, interests, budget, and available time. However, it did not calculate the actual travel time required to move between attractions or organize activities across multiple days. This meant that an itinerary could satisfy the basic preferences without fully considering whether the selected attractions were practical to visit together.

To address these limitations, I implemented a new planner that evaluates different combinations of attractions while checking several constraints at the same time. I also integrated OpenRouteService to obtain real walking and driving times and distances. These improvements allow MapOut to consider both the time spent visiting attractions and the time needed to travel between them.

The main focus of this week was therefore not to expand the dataset or change the existing project structure, but to improve the planning logic and test whether the generated itineraries satisfy the selected constraints.

1. Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem is a problem in which a solution must satisfy several conditions at the same time. Instead of choosing an option based on only one requirement, a CSP considers all the defined constraints before accepting a solution.

This approach is suitable for MapOut because planning a trip involves several connected decisions. For example, an attraction might match the user's interests and fit within the budget, but visiting it could require too much travel time. Therefore, selecting attractions based only on their categories or prices is not enough to create a practical itinerary.

A CSP is commonly described using variables, domains, and constraints. Variables represent the decisions that need to be made, domains contain the possible values for those decisions, and constraints define which combinations are allowed.

1.1 Applying Variables, Domains, and Constraints

In MapOut, the main decisions involve choosing which attractions to visit and assigning them to different days of the trip. The possible choices come from the attractions stored in the CSV dataset.

Before planning begins, the program filters the available attractions according to the selected destination and interests. This reduces the number of attractions that need to be considered during the search.

The planner then checks several constraints. The total estimated cost must remain within the user's budget, and the combined visiting and travel time must fit within the available time for each day. The travel time between consecutive attractions must also remain within the selected maximum transfer time.

Another important condition is that the same attraction cannot appear more than once in the itinerary. This prevents the planner from selecting an attraction again on a different day.

By checking these conditions together, MapOut can reject combinations that might look suitable individually but would not produce a valid travel plan.

1.2 Backtracking Search

To solve the planning problem, I implemented a backtracking search algorithm. Backtracking works by building a possible solution step by step and exploring different choices.

In MapOut, the algorithm considers an attraction and checks whether it can be added to the current itinerary without violating the constraints. If the attraction is suitable, the search continues with the next possible choice. If the combination is not suitable, the algorithm can return to an earlier decision and explore another possibility.

For example, if adding another attraction would exceed the remaining daily time or require a transfer longer than the allowed limit, that combination is rejected. The algorithm can then consider a different attraction instead.

This is useful because the order and combination of attractions affect the final itinerary. Two attractions might individually fit within the available time, but the travel time between them could make their combination unsuitable.

To prevent the algorithm from exploring an excessively large number of combinations, I also introduced limits on the number of candidate attractions and search states. These limits make the planning process more manageable, although they mean the program does not always guarantee the best possible itinerary.

2. CSP Implementation in MapOut

I implemented the CSP planner in src/csp_planner.py, keeping it separate from the baseline planner developed in Week 5.

The planner receives the user's destination, interests, budget, number of days, available time per day, transportation mode, and maximum transfer time. Based on these inputs, it first filters the dataset to find attractions that match the selected destination and interests.

After filtering, the backtracking algorithm explores different combinations of attractions. During this process, it checks whether each attraction can be added without exceeding the remaining budget, daily time limit, or maximum transfer time. It also keeps track of previously selected attractions to avoid duplicates.

The planner uses travel information from the routing module to calculate the time required between consecutive attractions. Once the search is completed, it returns a structured itinerary containing the selected attractions for each day, their visiting times, travel times, estimated costs, and distances.

```
# Check the daily limit using visiting time and real travel time
if details["minutes"] > minutes_per_day:
    continue

# Add the attraction and explore the next possibility
day.append(index)
backtrack(index + 1, total_cost + cost)

# Remove the attraction to try a different assignment
day.pop()
```

2.2 Generating Multi-Day Itineraries

One of the main improvements introduced was support for multi-day travel planning. Instead of producing a single list of attractions, the new planner organizes the selected activities into separate days. Each day has its own available time limit, while the overall trip shares the same total budget.

When assigning attractions, the algorithm considers the time needed to visit each location and the time required to travel between consecutive attractions. This helps prevent a day from containing more activities than the user has time to complete. The planner also tracks which attractions have already been included so that they are not repeated on another day.

After generating the itinerary, it displays the activities for each day and calculates a trip summary. The summary includes the total number of attractions, estimated cost, visiting time, travel time, and distance.

3. Real Travel Time and Distance Calculation

An important limitation of the previous planner was that it did not consider the actual routes between attractions. Two locations could be geographically close but still require more time to reach because of the available roads or walking paths.

To improve this, I integrated the OpenRouteService API, which provides estimated travel times and distances based on real routes.

I implemented this functionality in **src/route_matrix.py**. The module requests a route matrix containing travel information between the attractions being considered. Each entry represents the estimated time and distance required to travel from one attraction to another.

The CSP planner uses this information when evaluating possible combinations. For example, if the user selects driving and sets a maximum transfer time of 20 minutes, the planner checks whether the estimated driving time between consecutive attractions stays within that limit.

The application currently supports both walking and driving. This is important because the same two attractions can have different travel times depending on the selected transportation mode.

```
# Select the correct API profile for walking or driving
profile = PROFILES[transport_mode]

url = (
    "https://api.openrouteservice.org"
    f"/v2/matrix/{profile}"
)

# Prepare the request with coordinates and required measurements
request_data = json.dumps({
    "locations": coordinates,
    "metrics": ["distance", "duration"],
    "units": "km"
}).encode("utf-8")
```

3.1 Route Caching

I also implemented caching to avoid requesting the same route information repeatedly.

When the application calculates a route matrix, the result is saved locally. If the same route information is needed again, the program can reuse the stored result instead of making another API request. This reduces unnecessary requests and makes repeated testing more efficient.

4. Implementation and Testing

After implementing the CSP planner and routing functionality, I tested the different components to verify that they worked correctly together.

The project uses separate Python modules for loading the dataset, validating attraction information, generating itineraries, and calculating travel routes. I also kept the Week 5 baseline planner so that the previous functionality remains available for comparison.

4.1 Dataset Validation

Before testing the new planner, I checked the attraction dataset using the existing validation module. The dataset contains 101 attractions across eight destinations in North Macedonia. The validator checks the stored information and reports any detected problems.

The final validation test confirmed that all 101 attractions were valid, with zero warnings.

4.2 Testing the CSP Planner

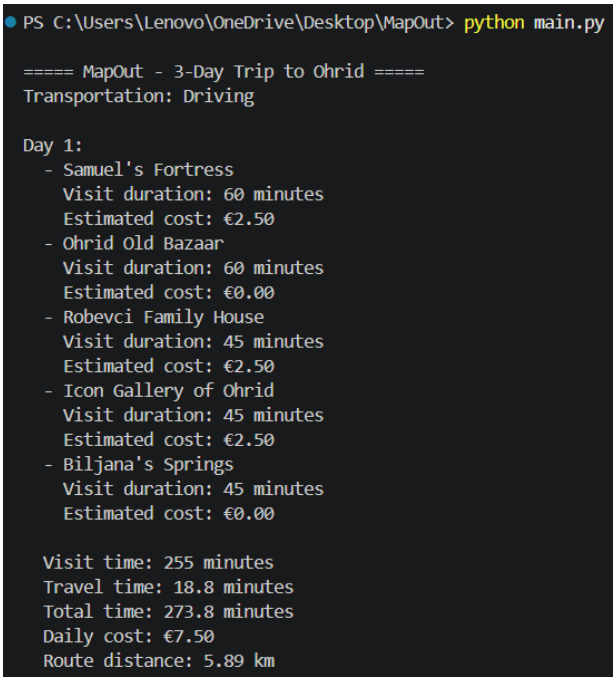
I tested the new planner by generating a three-day itinerary for Ohrid. The selected preferences included a total budget of €50, six available hours per day, driving as the transportation mode, and a maximum transfer time of 20 minutes. The planner generated an itinerary containing seven attractions.

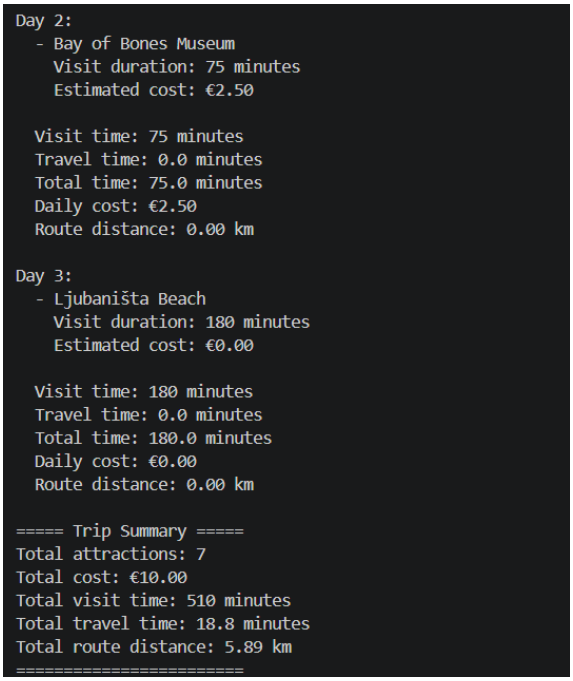
On the first day, it selected Samuel's Fortress, Ohrid Old Bazaar, Robevci Family House, Icon Gallery of Ohrid, and Biljana's Springs. These attractions required 255 minutes of visiting time and approximately 18.8 minutes of travel, giving a total planned duration of 273.8 minutes.

For the second day, the planner selected Bay of Bones Museum, while the third day included Ljubaništa Beach.

The total estimated attraction cost was €10, which remained below the selected €50 budget. The combined visiting time was 510 minutes, and all three days remained within their individual time limits.

The CSP test also confirmed that the selected attractions satisfied the implemented constraints and that attractions were not repeated.





4.3 Testing Real Routing

To verify the routing functionality, I tested travel between the Church of St. Nicholas in Sušica and the Vardar River Source at Vrutok.

For walking, the API returned an estimated distance of 13.09 km and a travel time of 157.1 minutes. For driving, the distance was approximately 12.85 km, with an estimated travel time of 20.4 minutes.

These results demonstrate why transportation mode is important when generating itineraries. Although the start and destination remain the same, the estimated travel time changes significantly depending on how the user plans to travel. Both routing tests passed successfully.

```
MapOut - Real Route Testing
From: Church of St. Nicholas, Sušica
To: Vardar River Source at Vrutok

Walking route:
Distance: 13.09 km
Estimated travel time: 157.1 minutes
Walking routing test passed!

Driving route:
Distance: 12.85 km
Estimated travel time: 20.4 minutes
Driving routing test passed!
```

5. Current Limitations and Future Improvements

Although the new planner successfully generates multi-day itineraries, there are still some limitations that I plan to address in future development.

Currently, the travel calculations consider movement between consecutive attractions within each day. They do not yet include the journey from accommodation to the first attraction or the return journey at the end of the day. For this reason, a day containing only one attraction currently shows zero travel time.

Another limitation is that attractions with unknown costs are not included when the planner requires a known estimated cost. This helps avoid exceeding the user's budget, but it also reduces the number of available choices.

The search also uses limits to keep the computation manageable. As a result, the generated itinerary satisfies the tested constraints but is not guaranteed to be the globally optimal combination of attractions.

In the following weeks, I plan to improve the planner by including accommodation, restaurants, and nearby locations. These additions would allow MapOut to generate more complete itineraries that better represent an actual trip.

Conclusion

During Week 6, I improved MapOut by implementing a CSP-based planner that uses backtracking to generate multi-day itineraries.

The main improvement is that the application now considers several constraints together instead of selecting attractions only through basic filtering. By combining the CSP planner with real travel time and distance information, MapOut can evaluate whether attractions are practical to visit within the available time and budget.

I also implemented route caching and tested the application using different scenarios. The successful results confirmed that the planner works with the implemented constraints and can generate structured itineraries for multiple days.

This week's work provides the foundation for further improvements, particularly the integration of accommodation and additional travel activities.

AI Assistance Disclosure

I used ChatGPT to help understand Constraint Satisfaction Problems and backtracking, develop the Python implementation, identify and resolve errors, organize the tests, and improve the explanations in this report.

I reviewed and adapted the suggested solutions to my MapOut project and tested the implementation locally.